



PERCONA
for PostgreSQL

Scaling PostgreSQL

Citus vs TimescaleDB

Choosing The Best-Fit

Overview

- whoami
- whoareu
- purpose of this presentation
- breakdown

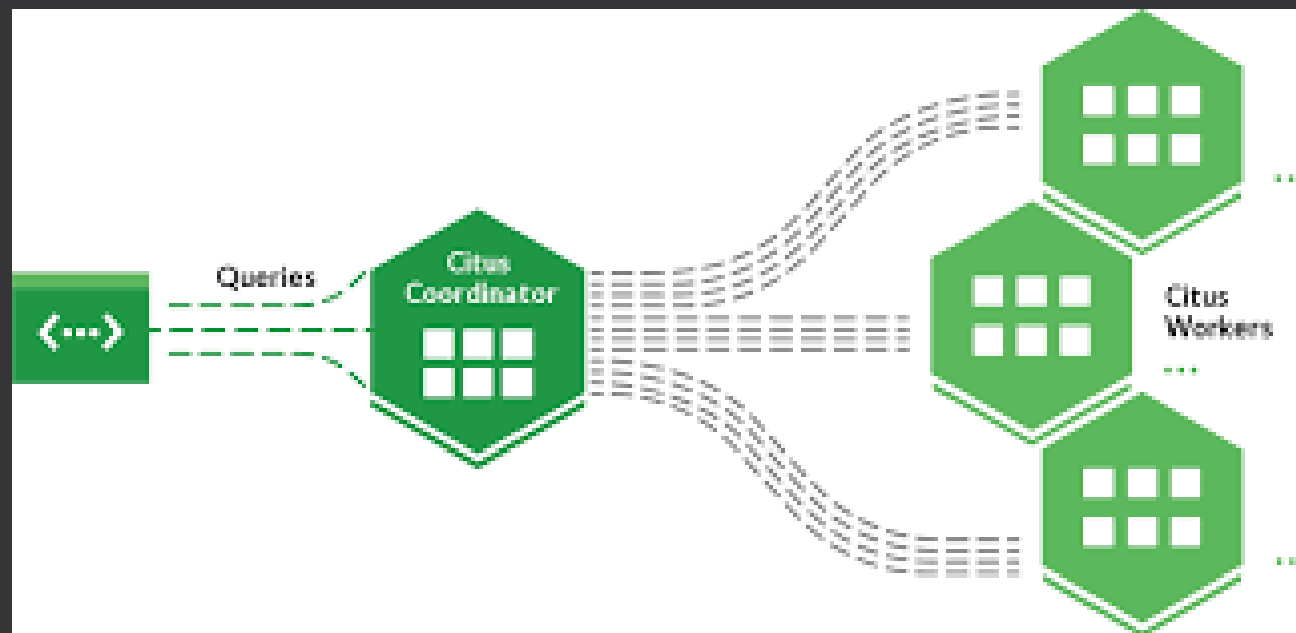
Breakdown

- What is Citus?
 - About Federated Databases
- What is TimescaleDB?
 - About Partitioning
- Working with Citus
- Working with Timescale
- Caveat and Conclusions



What is Citus?

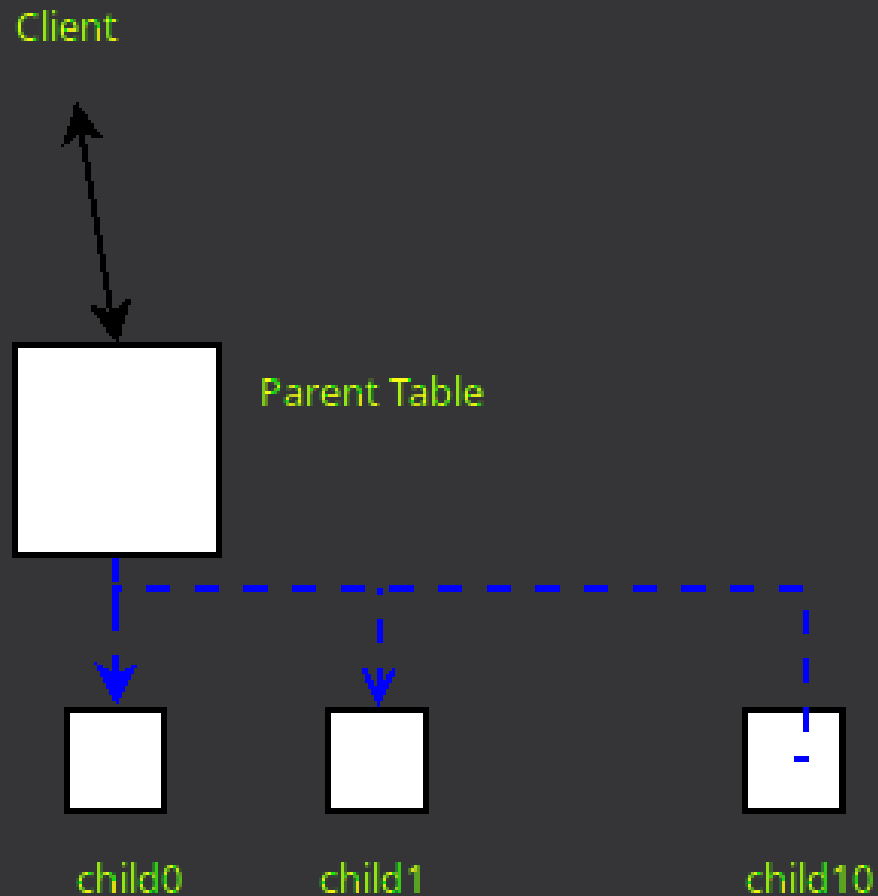
Federated database





What is Timescale?

Partitioned Tables





Working with
CitrusDB

CitusDB

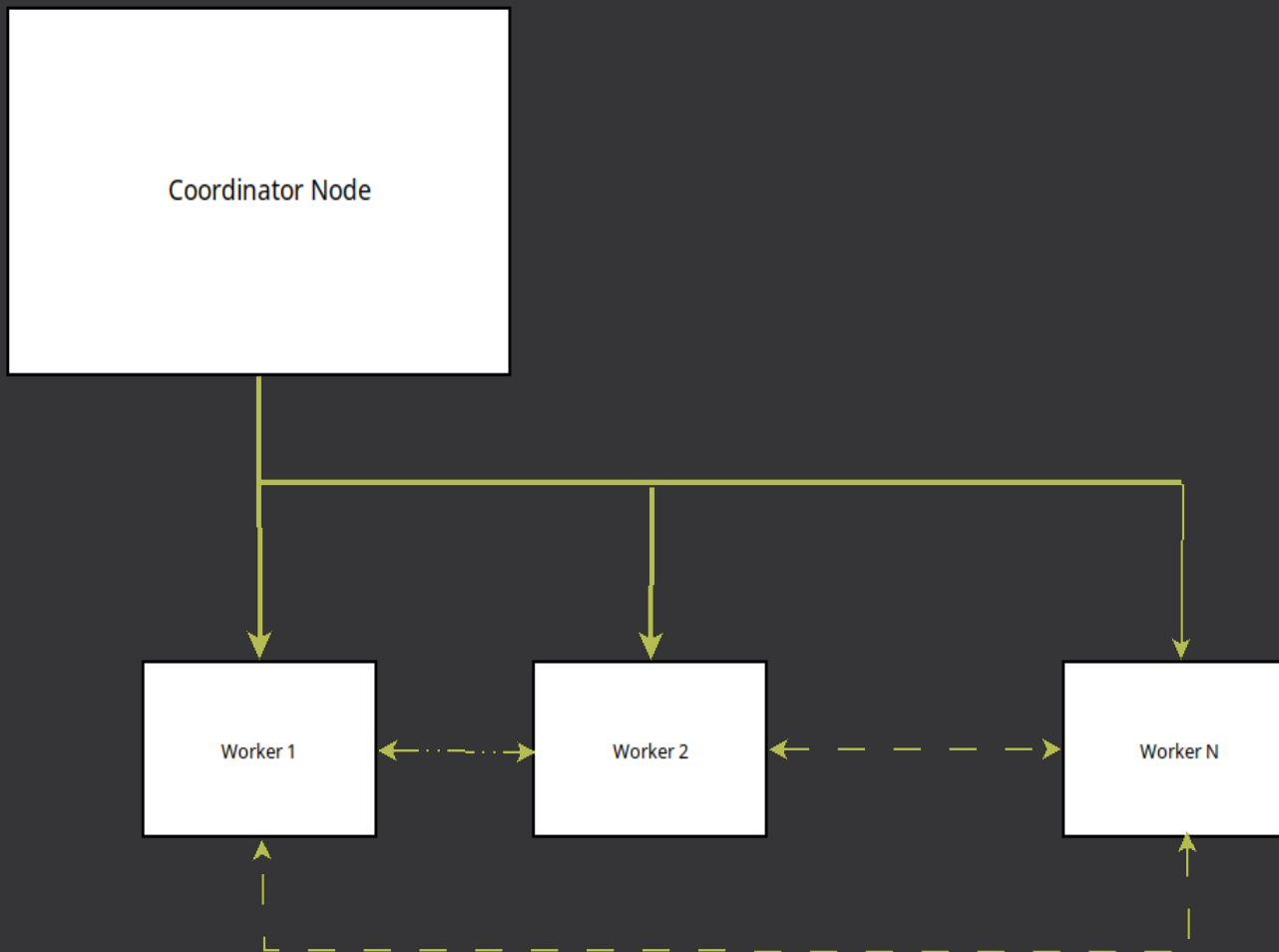
An extension that:

- horizontally scales PostgreSQL
- uses sharding and replication.
- Parallelizes SQL queries
- Can create column wise tables

Some CitusDB Concepts

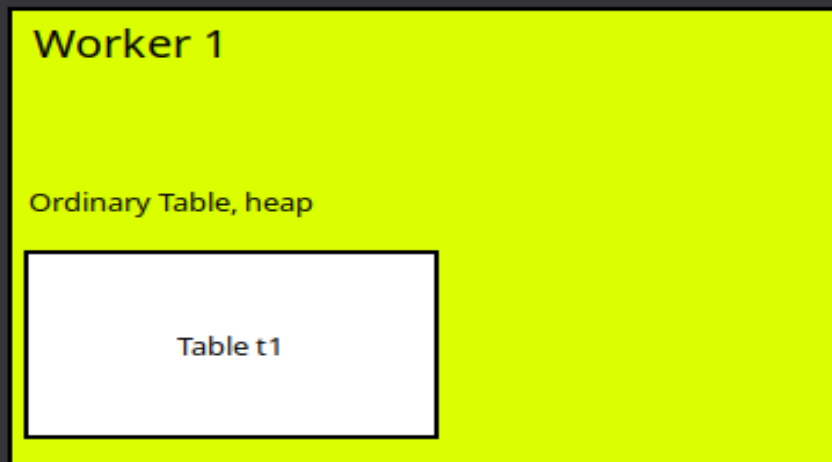
NODES

- coordinator
- worker



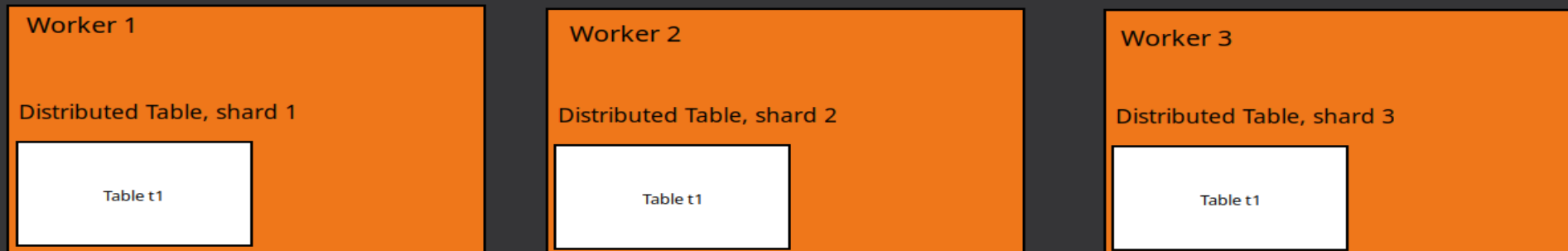
Some CitusDB Concepts

Ordinary Table: heap



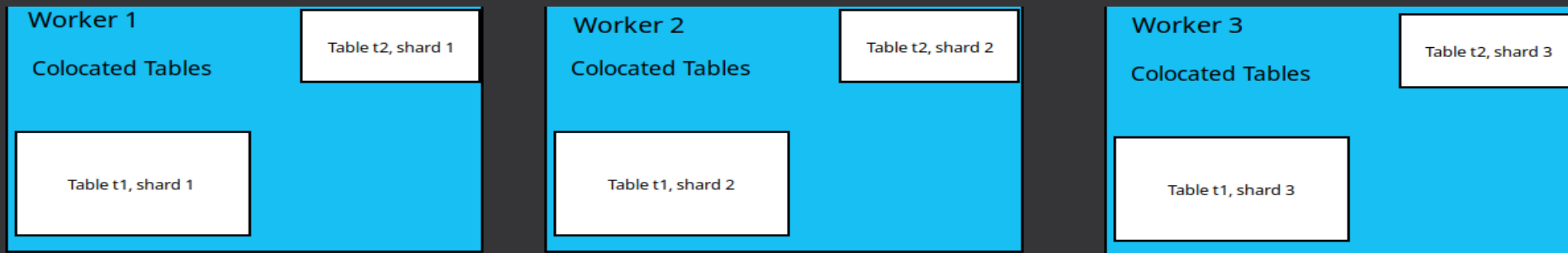
Some CitusDB Concepts

Distributed Table



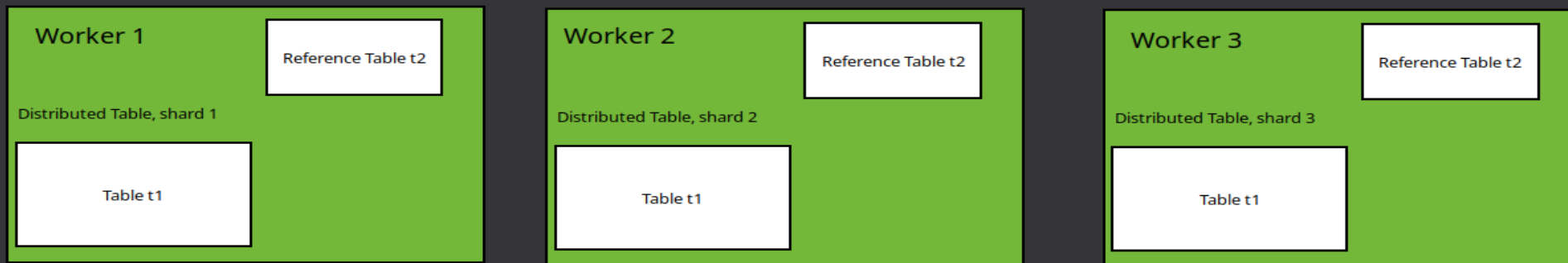
Some CitusDB Concepts

Distributed table, Colocated (foreign keys)



Some CitusDB Concepts

Reference Table (data redundancy)



SETUP

Ubuntu 24.04 LTS, noble

- INSTALL: via citus portal

- SCALE: The hosts
 - coordinator node
 - citus-coord

- worker nodes

- citus1

- citus2

- citus3

NAME	STATE	IPV4
citus1	RUNNING	10.108.134.217 (eth0)
citus2	RUNNING	10.108.134.38 (eth0)
citus3	RUNNING	10.108.134.216 (eth0)
citus-coord	RUNNING	10.108.134.213 (eth0)

Getting It

```
# CONFIGURE REPOSITORY
apt install -y wget curl gnupg2 locate mc man
curl https://install.citusdata.com/community/deb.sh | sudo bash
apt-get update && apt-get upgrade -y

# INSTALL THE SERVER
apt install -y postgresql-17-citus-14.0

# INSTALL CITUS MODULE
pg_conftool 17 main set shared_preload_libraries 'citus'

# CONFIGURE POSTGRES
pg_conftool 17 main set wal_level 'logical'
pg_conftool 17 main set listen_addresses '*'

# SERVER RESTART
systemctl restart postgresql@17-main
pg_lsclusters
```

Exploring the CitusDB Extension

```
-- login psql  
  
create database db01  
\c db01  
  
select *  
  from pg_available_extension_versions()  
  where name~'^citus'  
  order by 1,2;  
  
show shared_preload_libraries;  
  
create extension citus;  
  
select * from pg_extension;
```

Exploring the CitusDB Extension

About CitusDB Functions

```
select proname, prosrc
  from pg_proc
 where probin ~ 'citus'
 order by 1;

set
search_path=citus,citus_internal,columnar,columnar_internal;

db01=# \df
```

Schema	Name
citus_internal	acquire_citus_advisory_object_class_lock
citus_internal	add_colocation_metadata
citus_internal	add_object_metadata
citus_internal	add_partition_metadata
citus_internal	add_placement_metadata
citus_internal	add_shard_metadata
citus_internal	add_tenant_schema
citus_internal	adjust_local_clock_to_remote
citus_internal	citus_internal_copy_single_shard_placement
citus_internal	database_command
citus_internal	delete_colocation_metadata
citus_internal	delete_partition_metadata
citus_internal	delete_placement_metadata
citus_internal	delete_shard_metadata
citus_internal	delete_tenant_schema
citus_internal	find_groupid_for_node
citus_internal	global_blocked_processes
citus_internal	is_replication_origin_tracking_active
citus_internal	local_blocked_processes
citus_internal	mark_node_not_synced
citus_internal	pg_dist_node_trigger_func
citus_internal	pg_dist_rebalance_strategy_trigger_func
citus_internal	pg_dist_shard_placement_trigger_func
citus_internal	refresh_isolation_tester_prepared_statement
citus_internal	replace_isolation_tester_func
citus_internal	restore_isolation_tester_func
citus_internal	start_replication_origin_tracking
citus_internal	stop_replication_origin_tracking
citus_internal	unregister_tenant_schema_globally
citus_internal	update_none_dist_table_metadata
citus_internal	update_placement_metadata
citus_internal	update_relation_colocation

Exploring the CitusDB Extension

About CitusDB Schemas & Tables

```
db01=# \dn
```

```
          List of schemas
   Name          | Owner
-----+-----
 citus            | postgres
 citus_internal   | postgres
```

```
db01=# \d
```

```
          List of relations
 Schema | Name          | Type | Owner
-----+-----+-----+-----
 public | citus_schemas | view | postgres
 public | citus_tables   | view | postgres
```

TIP: use '\d+' to get detailed information about each relation

Exploring the CitusDB Extension About CitusDB Runtime Options

```
select *
  from pg_settings
 where name~'citus'
 order by 1,2;
```

name	setting
-----	-----
citus.all_modifications_commutative	off
citus.background_task_queue_interval	5000
citus.cluster_name	default
citus.coordinator_aggregation_strategy	row-gather
citus.count_distinct_error_rate	0
citus.cpu_priority	0
citus.cpu_priority_for_logical_replication_senders	inherit
citus.defer_drop_after_shard_move	on
citus.defer_drop_after_shard_split	on
citus.defer_shard_delete_interval	15000
citus.desired_percent_disk_available_after_move	10
citus.distributed_deadlock_detection_factor	2
citus.enable_binary_protocol	on
.	.
.	.
citus.stat_tenants_limit	100
citus.stat_tenants_log_level	off
citus.stat_tenants_period	60
citus.stat_tenants_track	none
citus.stat_tenants_untracked_sample_rate	1
citus.task_assignment_policy	greedy
citus.task_executor_type	adaptive
citus.use_citus_managed_tables	off
citus.use_secondary_nodes	never
citus.values_materialization_threshold	100
citus.version	12.1.1
citus.worker_min_messages	notice
citus.writable_standby_coordinator	off

(65 rows)

Working With CitusDB Scenarios ...

Columnar Tables

CitusDB Columnar Tables

Row-wise vs Column-wise

About Columnar Tables

- Row-oriented systems:
- PostgreSQL (default row-wise configuration)
- A column-oriented DBMS (columnar DBMS):

CitusDB Columnar Tables

Row-wise vs Column-wise

Introducing The Citus Columnar Extension

The CITUS columnar feature set includes:

- Highly compressed tables
- Projection Pushdown
- Chunk Group Filtering.

CitusDB Columnar Tables

```
-- Time: 8282.706 ms (00:08.283)
create table if not exists t1(id,qty)
using heap
as
select (random()*10)::int, (random()*1000)::int from generate_series(1,10e6);
```

```
-- Time: 4078.320 ms (00:04.078)
create table if not exists t2(id,qty)
using columnar
as
select (random()*10)::int, (random()*1000)::int from generate_series(1,10e6);
```

```
-- \d+ t?
```

Table "public.t1"									
Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description	
id	integer				plain				
qty	integer				plain				

Access method: heap

Table "public.t2"									
Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description	
id	integer				plain				
qty	integer				plain				

Access method: columnar

CitusDB Columnar Tables

A Working Example, cont'd

```
-- \dt+ t?
```

List of relations							
Schema	Name	Type	Owner	Persistence	Access method	Size	Description
public	t1	table	postgres	permanent	heap	342 MB	
public	t2	table	postgres	permanent	columnar	26 MB	

CitusDB Columnar Tables

SQL Statement Query Plans, Part I

SQL	Timings
-- HEAP TABLE explain analyze select id,qty from t1;	1204.1 ms
-- COLUMNAR TABLE explain analyze select id,qty from t2;	1448.036 ms
-- HEAP TABLE explain analyze select id,qty from t1 order by random();	10265.894 ms
-- COLUMNAR TABLE explain analyze select id,qty from t2 order by random();	10789.348 ms
-- HEAP TABLE explain analyze select sum(qty) from t1;	553.483 ms
-- COLUMNAR TABLE explain analyze select sum(qty) from t2;	1552.351 ms
-- HEAP TABLE explain analyze select id,sum(qty) from t1 group by id;	962.594 ms
-- COLUMNAR TABLE explain analyze select id,sum(qty) from t2 group by id;	2593.904 ms

10 million records, 2 column tables

Using the aforementioned table definitions, the following metrics were generated with the runtime parameter

max_parallel_workers_per_gather = 2

It's quite evident that, at least for these two tables, there's no performance benefit of a columnar accessed table over a regular heap accessed one

CitusDB Columnar Tables

SQL Statement Query Plans, Part II

```
create table t3 (  
c1 bigserial primary key  
,c2 bigint  
,c3 text default 'aowjfa fawjfawofjawofjaoifawevvaerarpfjkaofvaweawe[0JARG0IAJ0AFWF'  
,c4 text default 'aowjfa fawjfawofjawofjaoifawevvaerarpfjkaofvaweawe[0JARG0IAJ0AFWF'  
,  
,c99 text default 'aowjfa fawjfawofjawofjaoifawevvaerarpfjkaofvaweawe[0JARG0IAJ0AFWF'  
,c100 text default 'aowjfa fawjfawofjawofjaoifawevvaerarpfjkaofvaweawe[0JARG0IAJ0AFWF'  
) using heap;
```

```
create table t4(  
c1 bigserial primary key  
,c2 bigint  
,c3 text default 'aowjfa fawjfawofjawofjaoifawevvaerarpfjkaofvaweawe[0JARG0IAJ0AFWF'  
,c4 text default 'aowjfa fawjfawofjawofjaoifawevvaerarpfjkaofvaweawe[0JARG0IAJ0AFWF'  
,  
,c99 text default 'aowjfa fawjfawofjawofjaoifawevvaerarpfjkaofvaweawe[0JARG0IAJ0AFWF'  
,c100 text default 'aowjfa fawjfawofjawofjaoifawevvaerarpfjkaofvaweawe[0JARG0IAJ0AFWF'  
) using columnar;
```

```
--  
insert into t3(c2) select generate_series(1,5e6)*random()*10;  
--  
create index on t3(c2);  
--  
insert into t4(c2) select generate_series(1,5e6)*random()*10;  
create index on t4(c2);
```

Although the number of records is halved, the number of columns is increased from two to one hundred.

CitusDB Columnar Tables

SQL Statement Query Plans, Part II

List of relations							
Schema	Name	Type	Owner	Persistence	Access method	Size	Description
public	t3	table	postgres	permanent	heap	67 GB	
public	t4	table	postgres	permanent	columnar	89 MB	

				List of relations				
Schema	Name	Type	Owner	Table	Persistence	Access method	Size	Description
public	t3_c2_idx	index	postgres	t3	permanent	btree	105 MB	
public	t3_length_idx	index	postgres	t3	permanent	btree	33 MB	
public	t3_pkey	index	postgres	t3	permanent	btree	107 MB	
public	t4_c2_idx	index	postgres	t4	permanent	btree	105 MB	
public	t4_length_idx	index	postgres	t4	permanent	btree	33 MB	
public	t4_pkey	index	postgres	t4	permanent	btree	107 MB	

The columnar table's resultant compression is remarkable as the default size is reduced by a factor of 752X.

CitusDB Columnar Tables

SQL Statement Query Plans, Part II

SQL	Timings	
	Workers Per Gather 4	1
-- HEAP TABLE without index explain analyze select sum(c2) from t3;	9.6s	8.7s
-- COLUMNAR TABLE without index explain analyze select sum(c2) from t4;	590.176ms	596.459ms
-- HEAP TABLE explain analyze select count(c3) from t3;	10.4s	8.8s
-- COLUMNAR TABLE explain analyze select count(c3) from t4;	509.209ms	541.452ms
-- HEAP TABLE explain analyze select max(length(c25)) from t3;	1m34s	1m17s
-- COLUMNAR TABLE explain analyze select max(length(c25)) from t4;	1.1s	1.2s
-- HEAP TABLE explain analyze select sum(length(c50)) from t3;	1m33s	1m18s
-- COLUMNAR TABLE explain analyze select sum(length(c50)) from t4;	1.2s	1.2s

5 million records, 100 column tables

In order to get a better idea of performance differences a second set of tables, at a greater scale were created.

However this time, while the number of records was halved, the number of columns was increased from two to one hundred.

Even if most of the columns are simply copies of one another, the columnar table's resultant compression is remarkable as the default size is reduced by a factor of **752X!**

Unlike the first set of query plans, these ones clearly demonstrate a significant performance improvement.

Curious to see what would change in the way of performance, the varying the **max_parallel_workers_per_gather** doesn't appear to have changed much.

CitusDB Columnar Tables

Working with Indexes

SQL	Timings max parallel workers	
	4	1
-- HEAP TABLE using BTREE index explain analyze select sum(c2) from t3;	467.789ms	748.939ms
-- COLUMNAR TABLE using BTREE index explain analyze select sum(c2) from t4;	561.522ms	599.629ms
-- HEAP TABLE using EXPRESSION index explain analyze select max(length(c90)) from t3;	1.614ms	2.346ms
-- COLUMNAR TABLE using EXPRESSION index explain analyze select max(length(c90)) from t4;	31.980ms	38.766ms

Regarding The Relevance of Indexes

Because btree indexes of the same size are applied to both HEAP and columnar table's field, ie "c2" the resultant performance characteristics are basically the same but with the columnar table's index is slightly slower due to the extra processing required to uncompress the table's values.

The differences in performance were distinct when the less commonly used EXPRESSION index was used. Evidently these indexes are less efficient on columnar tables than the more mature HEAP access tables:

```
-- Expression indexes are created
create index on t3(length(c90));
create index on t4(length(c90));

-- example
select max(length(c90)) from t[34];
```

Regarding The Relevance Of Runtime Parameters

Unlike the results shown in the previous table, there were some changes querying a column with an index.

The btree index performance on the columnar table was more consistent than the HEAP table's

```
select sum(c2) from t[34];
```

CitusDB Columnar Tables

About DML Operations & Table Constraints

```
-- RECALL, create a new database and execute the following:
```

```
create table if not exists t1(id,qty)
using heap as select (random()*10)::int, (random()*1000)::int from generate_series(1,10e6);

create table if not exists t2(id,qty)
using columnar as select (random()*10)::int, (random()*1000)::int from generate_series(1,10e6);
```

```
-- fails
delete from t2 where id=5;
ERROR: UPDATE and CTID scans not supported for ColumnarScan
```

```
-- fails
update t2 set id=5;
ERROR: UPDATE and CTID scans not supported for ColumnarScan
```

```
-- Creating indexes on a columnar table is restricted
-- to btree indexes
```

```
-- works
create index on t2 using btree (id);
```

```
-- fails
create index on t2 using columnar (id);
ERROR: unsupported access method for the index on columnar table t2
```

Creating foreign key constraints aren't implemented

```
-- create table "t3"
select generate_series as id into t3 from generate_series(0,15);
alter table t3 add primary key(id);
```

```
-- works for our standard table t1
alter table t1 add foreign key(id) references t3(id);
```

```
-- fails with the columnar table t2
alter table t2 add foreign key(id) references t3(id);
ERROR: Foreign keys and AFTER ROW triggers are not supported for columnar tables
HINT: Consider an AFTER STATEMENT trigger instead.
```

```
--works after converting table t1 from COLUMNAR to HEAP
alter table t2 set access method heap;
alter table t2 add foreign key(id) references t3(id);
alter table t2 set access method columnar;
```

CitusDB Columnar Tables

Partitioning

Columnar tables can be used as partitions; a partitioned table can be made up of any combination of row and columnar partitions.

An excellent use case are INSERT once and READ only table partitions where one can leverage both its compression and better performing OLAP type queries for very large tables.

CitusDB Columnar Tables

Caveat, Reasons to use it

CITUS Columnar Extensions used when:

- There are “many” columns
- Space is at a premium
- The typical row is byte “heavy”
- OLAP is a major component of overall activity i.e. lots of different kinds of SELECT statements.
- INSERT performance is not a priority.

TIP, In regards to Partitioning:

- A partitioned table can be made up of any combination of row and columnar partitions.
- An excellent use case are INSERT once and READ only table partitions.
- Leverages its compression over very large tables.

REFERENCE:

<https://github.com/citusdata/citus/blob/main/src/backend/columnar/README.md>

Scaling A CitusDB Cluster

About The Cluster

SETUP

- postgres version 17
- citus version 14
- postgresql.conf configured
- pg_hba.conf configured
- postgres password: postgres
- .pgpass already configured for remote logins

Working With Data Redundancy

Working With Data Redundancy

Data Redundancy

- A condition created where the same piece of data is held in multiple locations
 - Basically a database version of a RAID
-

Advantages

- No Standby hosts
 - No fail-overs
 - No downtime
 - No backups
-

Disadvantages

- X-times nodes
- Significant disk space requirements
- SQL statement limitations (KISS)

Working With Data Redundancy

citus1	citus2	citus3	citus4
replica2x_102072	replica2x_102072	replica2x_102073	replica2x_102074
replica2x_102075	replica2x_102073	replica2x_102074	replica2x_102075
replica2x_102076	replica2x_102076	replica2x_102077	replica2x_102078
replica2x_102079	replica2x_102077	replica2x_102078	replica2x_102079
replica2x_102080	replica2x_102080	replica2x_102081	replica2x_102082
replica2x_102083	replica2x_102081	replica2x_102082	replica2x_102083
replica2x_102084	replica2x_102084	replica2x_102085	replica2x_102086
replica2x_102087	replica2x_102085	replica2x_102086	replica2x_102087
replica2x_102088	replica2x_102088	replica2x_102089	replica2x_102090
replica2x_102091	replica2x_102089	replica2x_102090	replica2x_102091
replica2x_102092	replica2x_102092	replica2x_102093	replica2x_102094
replica2x_102095	replica2x_102093	replica2x_102094	replica2x_102095
replica2x_102096	replica2x_102096	replica2x_102097	replica2x_102098
replica2x_102099	replica2x_102097	replica2x_102098	replica2x_102099
replica2x_102100	replica2x_102100	replica2x_102101	replica2x_102102
replica2x_102103	replica2x_102101	replica2x_102102	replica2x_102103

Data Redundancy, a database version of a RAID

Here is an example of a table named replica2x which has 2X redundancy across a cluster of nodes. The colors indicate specific shards of the table that are duplicated.

For example, if node citus1 goes offline, the sharded table it holds still has copies on nodes citus2 and citus4.

Likewise it can be said that if node citus2 goes offline the same data is still available on nodes 1,3,4.

CitusDB Data Redundancy

The CitusDB Cluster Setup

Method

- Step 1: set shard replication factor from 1X to 2X
- Step 2: create table `myevents2x` with 2X redundancy
- Step 3: identify and locate shards across `citus1` and `citus2`
- Step 4: identify some records to query from shards known to be on nodes `citus1` and `citus2`
- Step 5: test
 - shutdown `citus1`; perform afore identified query
 - startup `citus1`, shutdown `citus2`; perform afore identified query
 - restart `citus2`; perform afore identified query

CitusDB Data Redundancy

Step 1

```
# Execute as "postgres", in container citus-coord
#
for u in citus1 citus2 citus3 citus4 citus-coord
do
    echo "=== $u ==="
    dropdb -h $u -U postgres --if-exists db01
    createdb -h $u -U postgres db01
    psql -h $u -U postgres db01 -c 'create extension citus'
done
```

```
# Update shard replication factor from 1X to 2X

psql -h citus-coord db01 <<_eof_
    select citus_set_coordinator_host('citus-coord', 5432);

-- this INSERT is non-standard but it makes it easy to see the shards
    insert into pg_dist_node(nodename)
    values ('citus1')
           ,('citus2')
           ,('citus3')
           ,('citus4');

    alter system set citus.shard_replication_factor=2;
    select pg_reload_conf();
_eof_
```

```
# validate
psql -h citus-coord db01 <<_eof_
    show citus.shard_replication_factor;
    select nodeid,nodename,groupid,isactive from pg_dist_node order by 1;
_eof_
```


CitusDB Data Redundancy

Step 2

Create a new table with 2X redundancy

```
# execute on host:citus-coord, database db01

psql 'host=citus-coord dbname=db01 user=postgres' <<_eof_

create table myevents2x (
    device_id bigint,
    event_id bigserial,
    event_time timestamptz default now(),
    data jsonb not null,
    primary key (device_id, event_id)
);

-- distribute the events among the nodes
select create_distributed_table('myevents2x', 'device_id');

-- confirm table has been added across the cluster
select * from master_get_active_worker_nodes() order by 1;

-- populate the table
insert into myevents2x (device_id, data)
select s % 100, ('{"measurement":'||random()||'}')::jsonb
from generate_series(1,1000000) s;
_eof_
```

CitusDB Data Redundancy

Step 3

Locate a shard common on nodes citus1 and citus2

ex: myevents2x_102020 (numbers can be different)

```
# execute the following on host citus-coord
for u in citus1 citus2 citus3 citus4
do
    echo "==== $u ===="
    psql -h $u -U postgres db01 -c "select tablename from pg_tables where tablename~'myevents2x' order by 1"
done | less -S
```

CitusDB Data Redundancy

Step 3 cont'd

Citus 1	Citus 2
myevents2x_102008 myevents2x_102011 myevents2x_102012 myevents2x_102015 myevents2x_102016 myevents2x_102019 myevents2x_102020 myevents2x_102023 myevents2x_102024 myevents2x_102027 myevents2x_102028 myevents2x_102031 myevents2x_102032 myevents2x_102035 myevents2x_102036 myevents2x_102039	myevents2x_102008 myevents2x_102009 myevents2x_102012 myevents2x_102013 myevents2x_102016 myevents2x_102017 myevents2x_102020 myevents2x_102021 myevents2x_102024 myevents2x_102025 myevents2x_102028 myevents2x_102029 myevents2x_102032 myevents2x_102033 myevents2x_102036 myevents2x_102037

CitusDB Data Redundancy

Step 4

Locate and return the first three records
of a shard located on nodes citus1 and citus2

```
# execute the following on host citus-coord
for u in citus1 citus2
do
  SHARD="citus.myevents2x_102020"
  SQL="select * from $SHARD order by 1,2 limit 3"
  echo "==== host:$u, shard:$SHARD ====="
  psql -h $u -U postgres db01 -c "$SQL"
done | less -S
```

```
==== host:citus1, shard:citus.myevents2x_102020 ====
device_id | event_id |          event_time          | data
-----+-----+-----+-----
        36 |        36 | 2026-04-11 18:01:36.104738+00 | {"measurement": 0.97238332313129}
        36 |       136 | 2026-04-11 18:01:36.104738+00 | {"measurement": 0.2798833842558539}
        36 |       236 | 2026-04-11 18:01:36.104738+00 | {"measurement": 0.5639759436388552}
(3 rows)
```

```
==== host:citus2, shard:citus.myevents2x_102020 ====
device_id | event_id |          event_time          | data
-----+-----+-----+-----
        36 |        36 | 2026-04-11 18:01:36.104738+00 | {"measurement": 0.97238332313129}
        36 |       136 | 2026-04-11 18:01:36.104738+00 | {"measurement": 0.2798833842558539}
        36 |       236 | 2026-04-11 18:01:36.104738+00 | {"measurement": 0.5639759436388552}
(3 rows)
```

CitusDB Data Redundancy

Step 4, cont'd

```
# execute this query on the coordinator, and you get the same results
```

```
psql 'host=citus-coord dbname=db01 user=postgres' <<_eof_
select * from citus.myevents2x where device_id=36 and event_id in (36,136,236) order by 1,2;
_eof_
```

device_id	event_id	event_time	data
36	36	2026-04-11 18:01:36.104738+00	{"measurement": 0.97238332313129}
36	136	2026-04-11 18:01:36.104738+00	{"measurement": 0.2798833842558539}
36	236	2026-04-11 18:01:36.104738+00	{"measurement": 0.5639759436388552}

CitusDB Data Redundancy

Step 5

Redundancy Test: shutdown node citus1

These next few steps demonstrate the ability to continuously query by returning those records found in shard myevents2x_102020.

```
# execute on citus1
systemctl stop postgresql@17-main
```

```
psql -h citus-coord -U postgres db01 <<_eof_
select * from citus.myevents2x where device_id=36 and event_id in (36,136,236) order by 1,2;
_eof_
```

WARNING: connection to the remote node citus1:5432 failed with the following error: server closed the connection unexpectedly

This probably means the server terminated abnormally before or while processing the request.

device_id	event_id	event_time	data
36	36	2026-04-11 18:01:36.104738+00	{"measurement": 0.97238332313129}
36	136	2026-04-11 18:01:36.104738+00	{"measurement": 0.2798833842558539}
36	236	2026-04-11 18:01:36.104738+00	{"measurement": 0.5639759436388552}

(3 rows)

CitusDB Data Redundancy

Step 5, cont'd

Redundancy Test: restart citus1 and shutdown citus2

```
# execute on citus1
systemctl start postgresql@16-main
```

```
# execute on citus2
systemctl stop postgresql@16-main
```

```
psql -h citus-coord -U postgres db01 <<_eof_
select * from myevents2x where device_id=36 and event_id in (36,136,236) order by 1,2;
_eof_
```

device_id	event_id	event_time	data
36	36	2026-04-11 18:01:36.104738+00	{"measurement": 0.97238332313129}
36	136	2026-04-11 18:01:36.104738+00	{"measurement": 0.2798833842558539}
36	236	2026-04-11 18:01:36.104738+00	{"measurement": 0.5639759436388552}

(3 rows)

CitusDB Data Redundancy

Step 5, cont'd

Redundancy Test: restart citus2

```
# execute on citus2
systemctl start postgresql@16-main

psql -h citus-coord -U postgres db01 <<_eof_
select * from citus.myevents2x where device_id=36 and event_id in (36,136,236) order by 1,2;
_eof_
```

device_id	event_id	event_time	data
36	36	2026-04-11 18:01:36.104738+00	{"measurement": 0.97238332313129}
36	136	2026-04-11 18:01:36.104738+00	{"measurement": 0.2798833842558539}
36	236	2026-04-11 18:01:36.104738+00	{"measurement": 0.5639759436388552}

CitusDB Data Redundancy

Caveat, Issues to Beware

- Limited to performing simple SQL statements
- Current feature risks being deprecated in future releases ...

CitusDB Horizontal Scaling

CitusDB Horizontal Scaling

The CitusDB Cluster Setup

Method

- Step 1: one worker node
 - without indexes
 - with indexes
- Step 2: two worker node
- Step 3: three worker node
 - add 3rd node
 - add Foreign Key constraints, ala reference tables

CitusDB Horizontal Scaling

Setup, Clean up

```
# EXECUTE ON COORDINATOR NODE
# purge the previous scenario

psql -h citus-coord db01 <<_eof_

-- remove the distributed table
drop table if exists citus.myevents2x;

-- remove all active worker nodes
select * from citus_remove_node('citus1',5432);
select * from citus_remove_node('citus2',5432);
select * from citus_remove_node('citus3',5432);
select * from citus_remove_node('citus4',5432);

-- validate;there shouldn't be any active worker nodes
select * from master_get_active_worker_nodes() order by 1;

-- reset the replication factor from 2x to 1x
alter system set citus.shard_replication_factor=1;
select pg_reload_conf();
_eof_
```

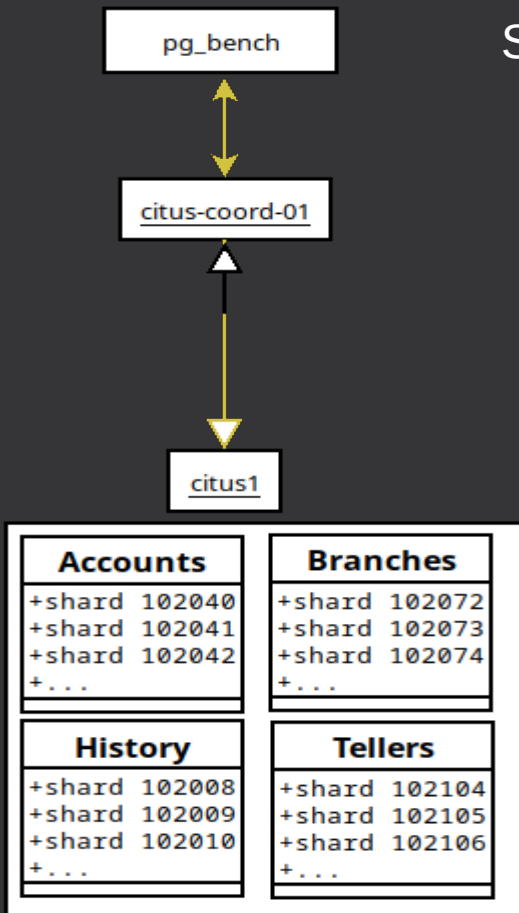
```
# EXECUTE ON COORDINATOR NODE

# drop db01
for u in citus-coord citus1 citus2 citus3 citus4
do
    echo "=== $u ==="
    psql "host=$u user=postgres password=postgres dbname=db01"<<_eof_
        drop extension if exists citus cascade;
    \c postgres
        drop database if exists db01;
_eof_
done

# create pgbench database
for u in citus-coord citus1 citus2 citus3 citus4
do
    echo "=== $u ==="
    psql "host=$u user=postgres password=postgres dbname=postgres"
<<_eof_
        create database pgbench;
    \c pgbench
        drop extension if exists citus cascade;
        create extension citus;
_eof_
done
```

CitusDB Horizontal Scaling

STEP 1: One Worker Node



```
psql -h citus-coord pgbench <<_eofl_
select citus_set_coordinator_host('citus-coord', 5432);
select citus_add_node('citus1', 5432);
_eofl_
```

```
# create only the tables
pgbench -h citus-coord -iI t pgbench
```

```
psql -h citus-coord pgbench -c '\dt+'
```

List of relations						
Schema	Name	Type	Owner	Persistence	Access method	Size
public	citus_tables	view	postgres	permanent		0 bytes
public	pgbench_accounts	table	postgres	permanent	heap	0 bytes
public	pgbench_branches	table	postgres	permanent	heap	0 bytes
public	pgbench_history	table	postgres	permanent	heap	0 bytes
public	pgbench_tellers	table	postgres	permanent	heap	0 bytes

```
# distributes the pgbench tables across node citus1
```

```
psql -h citus-coord pgbench <<_eofl_
BEGIN;
select create_distributed_table('pgbench_history', 'aid');
select create_distributed_table('pgbench_accounts', 'aid');
select create_distributed_table('pgbench_branches', 'bid');
select create_distributed_table('pgbench_tellers', 'tid');
COMMIT;
_eofl_
```

```
# populate the tables
pgbench -h citus-coord -iI g -s 300 pgbench
```

CitusDB Horizontal Scaling

One Worker Node, Adding Indexes

```
# ADD INDEXES
pgbench -h citus-coord -iI p pgbench

# Inspect Tables
for u in citus-coord citus1
do
    echo "===== $u ====="
    psql -h $u -U postgres pgbench -c '\d'
done | less -S
```

CitusDB Horizontal Scaling

One Worker Node, Benchmarking

A 60 Second benchmark, wo indexes

```
# execute the following on the coordinator node
pgbench -h citus-coord -c 20 -j 3 -T 60 -P 3 pgbench
```

```
-----
scaling factor: 300
query mode: simple
number of clients: 20
number of threads: 3
maximum number of tries: 1
duration: 60 s
number of transactions actually processed: 1570
number of failed transactions: 0 (0.000%)
latency average = 764.754 ms
latency stddev = 147.887 ms
initial connection time = 148.497 ms
tps = 26.084725 (without initial connection time)
```

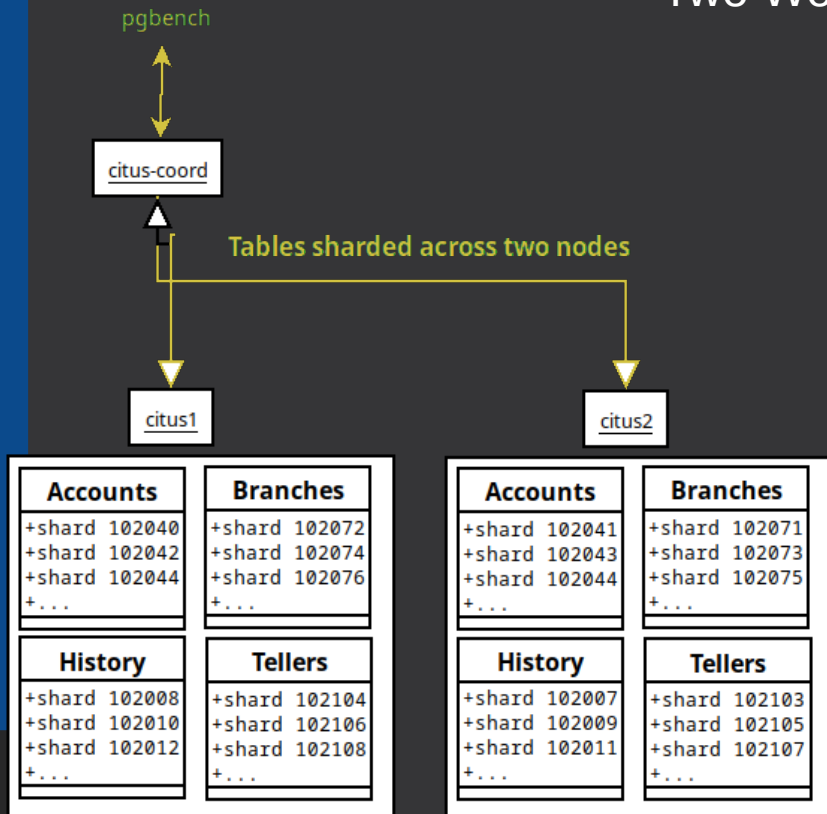
A 60 Second benchmark, with indexes

```
# execute the following on the coordinator node
pgbench -h citus-coord -c 20 -j 3 -T 60 -P 3 pgbench
```

```
-----
scaling factor: 300
query mode: simple
number of clients: 20
number of threads: 3
maximum number of tries: 1
duration: 60 s
number of transactions actually processed: 82479
number of failed transactions: 0 (0.000%)
latency average = 14.505 ms
latency stddev = 13.796 ms
initial connection time = 154.991 ms
tps = 1377.323730 (without initial connection time)
```

CitusDB Horizontal Scaling

Two Worker Node



```
psql -h citus-coord pgbench <<_eof_

-- ADD REPLICA IDENTITY
alter table public.pgbench_history REPLICA IDENTITY FULL;

-- ADD Node
select citus_add_node('citus2', 5432);

-- REBALANCE shards across two nodes
select * from rebalance_table_shards();

_eof_
```

```
-----
NOTICE: Moving shard 102009 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102013 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102029 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102034 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102020 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102015 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102011 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102019 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102023 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102038 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102024 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102017 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102028 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102026 from citus1:5432 to citus2:5432 ...
NOTICE: Moving shard 102022 from citus1:5432 to citus2:5432
-----
```

CitusDB Horizontal Scaling

CAVEAT

```
# FAILS
# - If logical replication NOT ENABLED (wal_level=logical)
# - Tables must have PK, UNIQ constraints or REPLICA IDENTITY
```

```
psql -h citus-coord pgbench <<_eof_
select * from rebalance_table_shards();
_eof_
```

```
ERROR: cannot use logical replication to transfer shards of the relation pgbench_history
since it doesn't have a REPLICA IDENTITY or PRIMARY KEY
```

```
DETAIL: UPDATE and DELETE commands on the shard will error out during logical replication
unless there is a REPLICA IDENTITY or PRIMARY KEY.
```

```
HINT: If you wish to continue without a replica identity set the shard_transfer_mode to
'force_logical' or 'block_writes'.
```



Working with TimescaleDB

Managing Time Series Data

TimescaleDB

An extension that optimizes PostgreSQL as a high performance time series database management system:

- 1) Enables, enhances and automates postgres table partitioning abstracted in a transparent manner
- 2) Provides customer SQL functions and table structures which is oriented towards:
 - 1) storage (i.e. high efficiency table/column compression)
 - 2) Has built-in time-series specific functions
 - 3) can schedule data retention policies
- 3) Provides continuous aggregates i.e. window functions
 - 1) able to sustain a high rate of inserts and deletes at scale
 - 2) in-depth OLAP analysis facilities

Some TimescaleDB Concepts

Examples of Time Series Data

- Monitoring computer systems:
 - CPU
 - free memory
 - net/disk IOPs
 - latency
- Financial trading systems:
 - payments
 - transactions
- Internet of things:
 - data from sensors
 - wearable devices
 - power meters
 - vehicles (telematics)
- Eventing applications
 - user/customer interaction
 - clickstreams
 - pageviews
- Environmental monitoring:
 - temperature
 - humidity
 - pressure
 - wind

Getting It

- 1) Timescale Portal
- 2) Github
- 3) Community Portal

Getting It

Community Portal

```
# CONFIGURE PG REPOSITORY
apt install -y postgresql-common
/usr/share/postgresql-common/pgdg/apt.postgresql.org.sh

# INSTALL THE SERVER
apt update && apt upgrade -y
apt install -y postgresql-17-timescaledb

# optional, nice to have ...
apt -y install net-tools mc locate man-db wget curl gnupg2 tree

# CONFIGURE POSTGRES
pg_conftool 17 main set wal_level 'logical'
pg_conftool 17 main set listen_addresses '*'
pg_conftool 17 main set shared_preload_libraries 'timescaledb'

# OPTIONAL
echo manual > /etc/postgresql/17/main/start.conf

# RESTART SERVER
systemctl restart postgresql@17-main
```

Getting It

Supplementary Packages not included

```
# Packages

timescaledb-2-loader-postgresql-17      # The loader for TimescaleDB

timescaledb-toolkit-postgresql-17      # A library of analytical hyperfunctions,
                                        # time-series pipelining, and other SQL
                                        # utilities

timescaledb-tools                       # A suite of tools that can be used with
                                        # TimescaleDB
                                        #   timescaledb-parallel-copy
                                        #   timescaledb-tune
```

Exploring the TimescaleDB Extension

```
postgres=# create database db01;
CREATE DATABASE
postgres=# \c db01
You are now connected to database "db01" as user "postgres".
db01=# create extension timescaledb;
CREATE EXTENSION
db01=# select * from pg_available_extensions() where name ~'timescale';
   name   | default_version | comment
-----+-----+-----
 timescaledb | 2.26.2         | Enables scalable inserts and complex queries for time-series data
(1 row)
```

Exploring the TimescaleDB Extension

About TimescaleDB Functions

```
-- 176 functions are returned
\setenv PAGER 'less -S'
select proname, prosrc from pg_proc where probin ~ 'timescale' order by 1;
```

prname	prosrc
add_columnstore_policy	ts_policy_compression_add
add_compression_policy	ts_policy_compression_add
add_continuous_aggregate_policy	ts_policy_refresh_cagg_add
add_dimension	ts_dimension_add_general
add_dimension	ts_dimension_add
add_job	ts_job_add
add_policies	ts_policies_add
add_process_hypertable_invalidations_policy	ts_policy_process_hyper_inval_add
add_reorder_policy	ts_policy_reorder_add
add_retention_policy	ts_policy_retention_add
alter_job	ts_job_alter
alter_job_set_hypertable_id	ts_job_alter_set_hypertable_id
alter_policies	ts_policies_alter
attach_chunk	ts_attach_chunk
attach_osm_table_chunk	ts_chunk_attach_osm_table_chunk
attach_tablespace	ts_tablespace_attach
bloom1_contains	ts_bloom1_contains
bloom1_contains_any	ts_bloom1_contains_any
bookend_deserializefunc	ts_bookend_deserializefunc
bookend_finalfunc	ts_bookend_finalfunc
bookend_serializefunc	ts_bookend_serializefunc
by_hash	ts_hash_dimension
by_range	ts_range_dimension
cagg_get_bucket_function_info	ts_continuous_agg_get_bucket_function_info
cagg_get_grouping_columns	ts_continuous_agg_get_grouping_columns
cagg_validate_query	ts_continuous_agg_validate_query

Exploring the TimescaleDB Extension

About TimescaleDB Schema

```
-- list of schemas in a timescale database  
\dn
```

List of schemas	
Name	Owner
-----+-----	
_timescaledb_cache	postgres
_timescaledb_catalog	postgres
_timescaledb_config	postgres
_timescaledb_debug	postgres
_timescaledb_functions	postgres
_timescaledb_internal	postgres
public	pg_database_owner

Exploring the TimescaleDB Extension

About TimescaleDB Functions

```
-- functions created in a timescale enabled database
-- 111 functions are returned
\setenv PAGER 'less -S'
set search_path=_timescaledb_cache,_timescaledb_catalog,_timescaledb_config,
               _timescaledb_debug,_timescaledb_functions,_timescaledb_internal;
\df
```

Schema	Name	
_timescaledb_functions	align_to_bucket	anyrange
_timescaledb_functions	alter_job_set_hypertable_id	integer
_timescaledb_functions	attach_osm_table_chunk	boolean
_timescaledb_functions	bloom1_contains	boolean
_timescaledb_functions	bloom1_contains_any	boolean
_timescaledb_functions	bloom1in	bloom1
_timescaledb_functions	bloom1out	cstring
_timescaledb_functions	bookend_deserializefunc	internal
_timescaledb_functions	bookend_finalfunc	anyelement
_timescaledb_functions	bookend_serializefunc	bytea
_timescaledb_functions	cagg_get_bucket_function_info	record
_timescaledb_functions	cagg_get_grouping_columns	text[]
_timescaledb_functions	cagg_validate_query	record
_timescaledb_functions	cagg_watermark	bigint
_timescaledb_functions	cagg_watermark_materialized	bigint
_timescaledb_functions	calculate_chunk_interval	bigint
_timescaledb_functions	chunk_constraint_add_table_constraint	void
_timescaledb_functions	chunk_id_from_relid	integer
_timescaledb_functions	chunk_rewrite_cleanup	
_timescaledb_functions	chunk_status	integer
_timescaledb_functions	chunk_status_text	text[]
_timescaledb_functions	chunk_status_text	text[]

Exploring the TimescaleDB Extension

About TimescaleDB Relations

```
-- tables created in a timescale enabled database
-- 27 tables are returned
\setenv PAGER 'less -S'
set search_path=_timescaledb_cache,_timescaledb_catalog,_timescaledb_config,
               _timescaledb_debug,_timescaledb_functions,_timescaledb_internal;
\dt
```

List of relations			
Schema	Name	Type	Owner
_timescaledb_cache	cache_inval_bgw_job	table	postgres
_timescaledb_cache	cache_inval_extension	table	postgres
_timescaledb_cache	cache_inval_hypertable	table	postgres
_timescaledb_catalog	chunk	table	postgres
_timescaledb_catalog	chunk_constraint	table	postgres
_timescaledb_catalog	chunk_index	table	postgres
_timescaledb_catalog	compression_algorithm	table	postgres
_timescaledb_catalog	compression_chunk_size	table	postgres
_timescaledb_catalog	compression_settings	table	postgres
_timescaledb_catalog	continuous_agg	table	postgres
_timescaledb_catalog	continuous_agg_migrate_plan	table	postgres
_timescaledb_catalog	continuous_agg_migrate_plan_step	table	postgres
_timescaledb_catalog	continuous_aggs_bucket_function	table	postgres
_timescaledb_catalog	continuous_aggs_hypertable_invalidation_log	table	postgres
_timescaledb_catalog	continuous_aggs_invalidation_threshold	table	postgres
_timescaledb_catalog	continuous_aggs_materialization_invalidation_log	table	postgres
_timescaledb_catalog	continuous_aggs_watermark	table	postgres
_timescaledb_catalog	dimension	table	postgres
_timescaledb_catalog	dimension_slice	table	postgres
_timescaledb_catalog	hypertable	table	postgres
_timescaledb_catalog	metadata	table	postgres
_timescaledb_catalog	tablespace	table	postgres
_timescaledb_catalog	telemetry_event	table	postgres
_timescaledb_config	bgw_job	table	postgres
_timescaledb_internal	bgw_job_stat	table	postgres
_timescaledb_internal	bgw_policy_chunk_stats	table	postgres
_timescaledb_internal	job_errors	table	postgres

(27 rows)

Exploring the TimescaleDB Extension

About TimescaleDB Runtime Parameters

```
-- 86 runtime parameters are returned
select * from pg_settings where name~'timescale' order by 1,2;
```

name	setting	unit	category	
timescaledb.auto_sparse_indexes	on		Customized Options	Create sparse indexes on compressed chunks
timescaledb.bgw_launcher_poll_time	60000		Customized Options	Launcher timeout value in milliseconds
timescaledb.bgw_log_level	warning		Customized Options	Log level for the background worker subsystem
timescaledb.bgw_scheduler_restart_time	-1	s	Customized Options	Restart time for the scheduler in seconds
timescaledb.compression_batch_size_limit	1000		Customized Options	The max number of tuples that can be batched
timescaledb.compression_orderby_default_function	_timescaledb_functions.get_orderby_defaults		Customized Options	Function that sets default orderby
timescaledb.compression_segmentby_default_function	_timescaledb_functions.get_segmentby_defaults		Customized Options	Function that sets default segmentby
timescaledb.compress_truncate_behaviour	truncate_only		Customized Options	Define behaviour of truncate after compress
timescaledb.debug_bgw_scheduler_exit_status	0		Customized Options	exit status to use when shutting down the s
timescaledb.debug_compression_path_info	off		Customized Options	show various compression-related debug info
timescaledb.debug_require_batch_sorted_merge	allow		Customized Options	require batch sorted merge in ColumnarScan
timescaledb.debug_skip_scan_info	off		Customized Options	Print debug info about SkipScan
timescaledb.default_chunk_time_interval			Customized Options	Default chunk time interval for new hypertable
timescaledb.direct_compress_copy_tuple_sort_limit	100000		Customized Options	Number of tuples that can be sorted at once
timescaledb.direct_compress_insert_tuple_sort_limit	10000		Customized Options	Number of tuples that can be sorted at once
timescaledb.disable_load	off		Customized Options	Disable the loading of the actual extension
timescaledb.enable_bool_compression	on		Customized Options	Enable bool compression functionality
timescaledb.enable_bulk_decompression	on		Customized Options	Enable decompression of the entire compressed
timescaledb.enable_cagg_create	on		Customized Options	Enable creation of continuous aggregate
timescaledb.enable_cagg_reorder_groupby	on		Customized Options	Enable group by reordering
timescaledb.enable_cagg_sort_pushdown	on		Customized Options	Enable sort pushdown for continuous aggregate
timescaledb.enable_cagg_watermark_constify	on		Customized Options	Enable cagg watermark constify
timescaledb.enable_cagg_window_functions	off		Customized Options	Enable window functions in continuous aggregate
timescaledb.enable_chunk_append	on		Customized Options	Enable chunk append node
timescaledb.enable_chunk_auto_publication	off		Customized Options	Enable automatic chunk publication
timescaledb.enable_chunk_skipping	off		Customized Options	Enable chunk skipping functionality
timescaledb.enable_chunkwise_aggregation	on		Customized Options	Enable chunk-wise aggregation
timescaledb.enable_columnarindexscan	on		Customized Options	Enable metadata-only optimization for Columnar
timescaledb.enable_columnarscan	on		Customized Options	Enable ColumnarScan for columnar storage
timescaledb.enable_composite_bloom_indexes	on		Customized Options	Enable creation of the bloom1 composite index

INTRODUCING THE TIMESCALEDDB TABLE

Creating The HyperTable

STEPS

- Create a regular table

```
create mytable (c1 int, c2 ##, ... t_stamp <time>)
```

- Convert the table into a HyperTable

```
select * from create_hypertable ('mytable','t_stamp')
```

Introducing the TimescaleDB Table

Populating The HyperTable

EXAMPLE #1 (database: ex01)
Partitioning policy set at default of 7 days

```
#!/bin/bash
#
# INVOCATION: Example 01, create and populate a Hypertable
#
# ./go.sh
#
set -e

dropdb --if-exists ex01
createdb ex01

psql ex01 <<_eof_
\set ON_ERROR_STOP on
create extension timescaledb;
create sequence increment_seq as bigint increment by 7200;
create table t1 (
    id bigserial,
    t_stamp timestamp with time zone not null default now(),
    payload text
);

-- create the hyper table
select * from create_hypertable('t1', 't_stamp');
_eof_

for u in $(seq 1 1e3)
do
    PAYLOAD=$(tr -dc A-Za-z0-9 </dev/urandom | head -c 1024000; echo)

    psql -q ex01 <<_eof_
    insert into t1(t_stamp,payload)
    select nextval('increment_seq')::text::interval+now(),
    '$PAYLOAD';
_eof_
```

```
ex01=# \d
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | increment_seq | sequence | postgres
public | t1 | table | postgres
public | t1_id_seq | sequence | postgres
(3 rows)

ex01=# select show_chunks('t1');
show_chunks
-----
timescaledb_internal._hyper_1_1_chunk
timescaledb_internal._hyper_1_2_chunk
timescaledb_internal._hyper_1_3_chunk
timescaledb_internal._hyper_1_4_chunk
timescaledb_internal._hyper_1_5_chunk
timescaledb_internal._hyper_1_6_chunk
timescaledb_internal._hyper_1_7_chunk
timescaledb_internal._hyper_1_8_chunk
timescaledb_internal._hyper_1_9_chunk
timescaledb_internal._hyper_1_10_chunk
timescaledb_internal._hyper_1_11_chunk
timescaledb_internal._hyper_1_12_chunk
timescaledb_internal._hyper_1_13_chunk
(13 rows)

ex01=# select * from pg_size_pretty(hypertable_size('t1'));
pg_size_pretty
-----
1016 MB
(1 row)
```

Introducing the TimescaleDB Table

Understanding HyperTable Chunks

```
ex01=# \d+ t1
```

Table "public.t1"									
Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description	
id	bigint		not null	nextval('t1_id_seq'::regclass)	plain				
t_stamp	timestamp with time zone		not null	now()	plain				
payload	text				extended				

```
Indexes:
```

```
"t1_t_stamp_idx" btree (t_stamp DESC)
```

```
Triggers:
```

```
ts_insert_blocker BEFORE INSERT ON t1 FOR EACH ROW EXECUTE FUNCTION _timescaledb_functions.insert_blocker()
```

```
Child tables:
```

```
_timescaledb_internal.hyper_1_10_chunk,  
_timescaledb_internal.hyper_1_11_chunk,  
_timescaledb_internal.hyper_1_12_chunk,  
_timescaledb_internal.hyper_1_13_chunk,  
_timescaledb_internal.hyper_1_1_chunk,  
_timescaledb_internal.hyper_1_2_chunk,  
_timescaledb_internal.hyper_1_3_chunk,  
_timescaledb_internal.hyper_1_4_chunk,  
_timescaledb_internal.hyper_1_5_chunk,  
_timescaledb_internal.hyper_1_6_chunk,  
_timescaledb_internal.hyper_1_7_chunk,  
_timescaledb_internal.hyper_1_8_chunk,  
_timescaledb_internal.hyper_1_9_chunk
```

```
Access method: heap
```

```
ex01=# \d _timescaledb_internal.hyper_1_1_chunk
```

Table "_timescaledb_internal.hyper_1_1_chunk"				
Column	Type	Collation	Nullable	Default
id	bigint		not null	nextval('t1_id_seq'::regclass)
t_stamp	timestamp with time zone		not null	now()
payload	text			

```
Indexes:
```

```
"_hyper_1_1_chunk_t1_t_stamp_idx" btree (t_stamp DESC)
```

```
Check constraints:
```

```
"constraint_1" CHECK (t_stamp >= '2024-03-21 00:00:00+00'::timestamp with time zone AND t_stamp < '2024-03-28 00:00:00+00'::timestamp with time zone)
```

```
Inherits: t1
```

Introducing the TimescaleDB Table

Understanding HyperTable Chunks

```
ex01=# \dt+
```

List of relations							
Schema	Name	Type	Owner	Persistence	Access method	Size	Description
public	t1	table	postgres	permanent	heap	8192 bytes	

(1 row)

```
ex01=# \di+
```

List of relations									
Schema	Name	Type	Owner	Table	Persistence	Access method	Size	Description	
public	t1_t_stamp_idx	index	postgres	t1	permanent	btree	8192 bytes		

(1 row)

```
ex01=# \di+ _timescaledb_internal.*_hyper*
```

List of relations										
Schema	Name	Type	Owner	Table	Persistence	Access method	Size	Description		
_timescaledb_internal	_hyper_1_10_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_10_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_11_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_11_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_12_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_12_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_13_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_13_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_1_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_1_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_2_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_2_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_3_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_3_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_4_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_4_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_5_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_5_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_6_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_6_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_7_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_7_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_8_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_8_chunk	permanent	btree	16 kB			
_timescaledb_internal	_hyper_1_9_chunk_t1_t_stamp_idx	index	postgres	_hyper_1_9_chunk	permanent	btree	16 kB			

(13 rows)

DML OPERATIONS

Introducing the TimescaleDB Table

SELECT

```
ex01=# select show_chunks('t1');
          show_chunks
-----
 _timescaledb_internal.hyper_1_1_chunk
 _timescaledb_internal.hyper_1_2_chunk
 _timescaledb_internal.hyper_1_3_chunk
 _timescaledb_internal.hyper_1_4_chunk
 _timescaledb_internal.hyper_1_5_chunk
 _timescaledb_internal.hyper_1_6_chunk
 _timescaledb_internal.hyper_1_7_chunk
 _timescaledb_internal.hyper_1_8_chunk
 _timescaledb_internal.hyper_1_9_chunk
 _timescaledb_internal.hyper_1_10_chunk
 _timescaledb_internal.hyper_1_11_chunk
 _timescaledb_internal.hyper_1_12_chunk
 _timescaledb_internal.hyper_1_13_chunk
(13 rows)
```

```
ex01=# explain analyze
select id,t_stamp
from t1
where t_stamp between '2024-05-09 00:00:00+00' and '2024-05-16';
          QUERY PLAN
-----
 Append  (cost=0.00..4.50 rows=63 width=16) (actual time=0.007..0.015 rows=62 loops=1)
   -> Seq Scan on _hyper_1_1_chunk  (cost=0.00..1.93 rows=62 width=16) (actual time=0.003..0.006 rows=62 loops=1)
        Filter: ((t_stamp >= '2024-05-09 00:00:00+00'::timestamp with time zone) AND (t_stamp <= '2024-05-16 00:00:00+00'::timestamp with time zone))
   -> Seq Scan on _hyper_1_2_chunk  (cost=0.00..2.26 rows=1 width=16) (actual time=0.003..0.003 rows=0 loops=1)
        Filter: ((t_stamp >= '2024-05-09 00:00:00+00'::timestamp with time zone) AND (t_stamp <= '2024-05-16 00:00:00+00'::timestamp with time zone))
        Rows Removed by Filter: 84
 Planning Time: 0.168 ms
 Execution Time: 0.023 ms
(8 rows)
```

Introducing the TimescaleDB Table

UPDATE

```
ex01=# explain analyze
update t1 set payload='updated record' where id=1 and t_stamp < '2024-05-16';
               QUERY PLAN
-----
Custom Scan (HypertableModify) (cost=0.00..1.94 rows=1 width=42) (actual time=0.699..0.699 rows=0 loops=1)
  -> Update on t1 (cost=0.00..1.94 rows=1 width=42) (actual time=0.699..0.699 rows=0 loops=1)
      Update on _hyper_1_1_chunk t1_1
      -> Result (cost=0.00..1.94 rows=1 width=42) (actual time=0.003..0.006 rows=1 loops=1)
          -> Seq Scan on _hyper_1_1_chunk t1_1 (cost=0.00..1.93 rows=1 width=10) (actual time=0.003..0.005 rows=1 loops=1)
              Filter: ((t_stamp < '2024-05-16 00:00:00+00':timestamp with time zone) AND (id = 1))
              Rows Removed by Filter: 61
Planning Time: 0.851 ms
Execution Time: 0.729 ms
(9 rows)
```

Introducing the TimescaleDB Table

DELETE

```
ex01=# explain analyze
delete from t1 where t_stamp::date='2024-05-15' and t_stamp < '2024-05-15';
                                QUERY PLAN
-----
Custom Scan (HypertableModify) (cost=0.00..2.08 rows=1 width=10) (actual time=0.043..0.044 rows=0 loops=1)
-> Delete on t1 (cost=0.00..2.08 rows=1 width=10) (actual time=0.043..0.044 rows=0 loops=1)
    Delete on _hyper_1_1_chunk t1_1
    -> Custom Scan (ChunkAppend) on t1 (cost=0.00..2.08 rows=1 width=10) (actual time=0.009..0.010 rows=0 loops=1)
        Chunks excluded during startup: 0
        -> Seq Scan on _hyper_1_1_chunk t1_1 (cost=0.00..2.08 rows=1 width=10) (actual time=0.009..0.009 rows=0 loops=1)
            Filter: ((t_stamp < '2024-05-15 00:00:00+00'::timestamp with time zone) AND ((t_stamp)::date = '2024-05-15'::date))
            Rows Removed by Filter: 62
Planning Time: 0.195 ms
Execution Time: 0.059 ms
(10 rows)
```

Introducing the TimescaleDB Table

INSERT

```
ex01=# -- look what happens when inserting out of order
insert into t1(id,t_stamp,payload) values(default,now()+ '1 year'+ '2 day','hello world');
insert into t1(id,t_stamp,payload) values(default,now()+ '1 year'+ '1 day','hello world');
insert into t1(id,t_stamp,payload) values(default,now()+ '1 year','hello world');
insert into t1(id,t_stamp,payload) values(default,now()+ '6 months'+ '2 day','hello world');
insert into t1(id,t_stamp,payload) values(default,now()+ '6 months'+ '1 day','hello world');
insert into t1(id,t_stamp,payload) values(default,now()+ '6 months','hello world');
```

```
ex01=# select show_chunks('t1');
        show_chunks
```

```
-----
 _timescaledb_internal.hyper_1_1_chunk
 _timescaledb_internal.hyper_1_2_chunk
 _timescaledb_internal.hyper_1_3_chunk
 _timescaledb_internal.hyper_1_4_chunk
 _timescaledb_internal.hyper_1_5_chunk
 _timescaledb_internal.hyper_1_6_chunk
 _timescaledb_internal.hyper_1_7_chunk
 _timescaledb_internal.hyper_1_8_chunk
 _timescaledb_internal.hyper_1_9_chunk
 _timescaledb_internal.hyper_1_10_chunk
 _timescaledb_internal.hyper_1_11_chunk
 _timescaledb_internal.hyper_1_12_chunk
 _timescaledb_internal.hyper_1_13_chunk
 _timescaledb_internal.hyper_1_14_chunk
 _timescaledb_internal.hyper_1_15_chunk
(15 rows)
```

```
select id,t_stamp from _timescaledb_internal.hyper_1_14_chunk;
```

id	t_stamp
1001	2025-05-12 20:57:45.973733+00
1002	2025-05-11 20:57:45.985099+00
1003	2025-05-10 20:57:45.988455+00

```
select id,t_stamp from _timescaledb_internal.hyper_1_15_chunk;
```

id	t_stamp
1004	2024-11-12 20:57:45.994969+00
1005	2024-11-11 20:57:46.005661+00
1006	2024-11-10 20:57:46.009228+00

Introducing the TimescaleDB Table

INSERT

REORDERING RECORDS within a CHUNK

```
-- note the PRIMARY KEY column order
alter table t1 add primary key(t_stamp,id);
```

```
select reorder_chunk('_timescaledb_internal._hyper_1_14_chunk', 't1_pkey');
select reorder_chunk('_timescaledb_internal._hyper_1_15_chunk', 't1_pkey');
```

```
ex01=# select id,t_stamp from _timescaledb_internal._hyper_1_14_chunk;
 id |          t_stamp
-----+-----
 1003 | 2025-05-10 21:39:08.277988+00
 1002 | 2025-05-11 21:39:08.274621+00
 1001 | 2025-05-12 21:39:08.247648+00
(3 rows)
```

```
ex01=# select id,t_stamp from _timescaledb_internal._hyper_1_15_chunk;
 id |          t_stamp
-----+-----
 1006 | 2024-11-10 21:39:08.298595+00
 1005 | 2024-11-11 21:39:08.295038+00
 1004 | 2024-11-12 21:39:08.281141+00
(3 rows)
```

REFERENCE

https://docs.timescale.com/api/latest/hypertable/reorder_chunk/
https://docs.timescale.com/api/latest/hypertable/add_reorder_policy/

ADMINISTERING: HYPERTABLE CHUNKS

Administering Hypertable CHUNKS

Introducing HyperTable Partitions

TimescaleDB automatically partitions its tables into **CHUNKS**.

CHUNKS

- Child tables that are partitioned by time
- Key column is automatically defined at table creation
- Default time intervals are set at 7 days
- Time intervals can be set to any value

CHUNK Administration

- Can be compressed
 - manually
 - by setting a policy (can be changed dynamically)
- Can be dropped:
 - manually
 - by setting a policy (can be changed dynamically)
- Can aggregate existing chunks forming larger interval sets
- Can be reordered by changing index order i.e. similar to CLUSTER
- Can be moved (to a different tablespace)

Best Practices

- **CHUNK Ideal size:** 1 CHUNK should be no larger than 25% of available RAM
- Compress older CHUNKs to save disk space
- Drop older CHUNKS when no longer required

Administering Hypertable CHUNKS

Data Population Over Time, a simulation

EXAMPLE (database: ex02)

```
dropdb --if-exists ex02
createdb ex02
\c ex02

create extension timescaledb;

-- create the table
create table t_timescale(
  id uuid
,c2 int default random()*1E6
,c3 int default random()*1E6
,c4 int default random()*1E6
,c5 int default random()*1E6
,c6 int default random()*1E6
,c7 int default random()*1E6
,t_stamp timestamptz not null default clock_timestamp()
);
```

```
-- convert to a hypertable
select create_hypertable('t_timescale', 't_stamp');

select *
  from set_chunk_time_interval('t_timescale', interval '1
minutes');

-- populate table with data
insert into t_timescale(id)
  select gen_random_uuid() from generate_series(1,5E7);
```

Administering Hypertable CHUNKS

Populating A HyperTable

EXAMPLE #2 (database: ex02)
Partitioning policy set at 1 minute.

```
#!/bin/bash
#
# INVOCATION: Example 02, administering a Hypertable
#
# ./go-ex02.sh
#
set -e

dropdb --if-exists ex02
createdb ex02

time \
psql ex02 <<_eof_
create extension timescaledb;

create table t_timescale(
id uuid
,c2 int default random()*1E6
,c3 int default random()*1E6
,c4 int default random()*1E6
,c5 int default random()*1E6
,c6 int default random()*1E6
,c7 int default random()*1E6
,t_stamp timestamptz not null default clock_timestamp()
);

select create_hypertable('t_timescale', 't_stamp');
select * from set_chunk_time_interval('t_timescale',interval '1 minutes');

insert into t_timescale(id) select gen_random_uuid() from generate_series(1,5E7);
_eof_
~
```

Administering Hypertable CHUNKS

```
ex02=# \dt+ _timescaledb_internal.*_hyper*
```

List of relations						
Schema	Name	Type	Owner	Persistence	Access method	Size
_timescaledb_internal	_hyper_1_1_chunk	table	postgres	permanent	heap	184 MB
_timescaledb_internal	_hyper_1_2_chunk	table	postgres	permanent	heap	587 MB
_timescaledb_internal	_hyper_1_3_chunk	table	postgres	permanent	heap	712 MB
_timescaledb_internal	_hyper_1_4_chunk	table	postgres	permanent	heap	677 MB
_timescaledb_internal	_hyper_1_5_chunk	table	postgres	permanent	heap	704 MB
_timescaledb_internal	_hyper_1_6_chunk	table	postgres	permanent	heap	703 MB
_timescaledb_internal	_hyper_1_7_chunk	table	postgres	permanent	heap	84 MB

(7 rows)

```
ex02=# \di+ _timescaledb_internal.*_hyper*
```

List of relations						
Schema	Name	Type	Owner	Table	Persistence	Access method
_timescaledb_internal	_hyper_1_1_chunk_t_timescale_t_stamp_idx	index	postgres	_hyper_1_1_chunk	permanent	btree
_timescaledb_internal	_hyper_1_2_chunk_t_timescale_t_stamp_idx	index	postgres	_hyper_1_2_chunk	permanent	btree
_timescaledb_internal	_hyper_1_3_chunk_t_timescale_t_stamp_idx	index	postgres	_hyper_1_3_chunk	permanent	btree
_timescaledb_internal	_hyper_1_4_chunk_t_timescale_t_stamp_idx	index	postgres	_hyper_1_4_chunk	permanent	btree
_timescaledb_internal	_hyper_1_5_chunk_t_timescale_t_stamp_idx	index	postgres	_hyper_1_5_chunk	permanent	btree
_timescaledb_internal	_hyper_1_6_chunk_t_timescale_t_stamp_idx	index	postgres	_hyper_1_6_chunk	permanent	btree
_timescaledb_internal	_hyper_1_7_chunk_t_timescale_t_stamp_idx	index	postgres	_hyper_1_7_chunk	permanent	btree

(7 rows)

```
ex02=# \d+ _timescaledb_internal._hyper_1_1_chunk
```

Table "_timescaledb_internal._hyper_1_1_chunk"									
Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description	
id	uuid				plain				
c2	integer			(random() * '1000000'::numeric::double precision)	plain				
c3	integer			(random() * '1000000'::numeric::double precision)	plain				
c4	integer			(random() * '1000000'::numeric::double precision)	plain				
c5	integer			(random() * '1000000'::numeric::double precision)	plain				
c6	integer			(random() * '1000000'::numeric::double precision)	plain				
c7	integer			(random() * '1000000'::numeric::double precision)	plain				
t_stamp	timestamp with time zone		not null	clock_timestamp()	plain				

Indexes:

"_hyper_1_1_chunk_t_timescale_t_stamp_idx" btree (t_stamp DESC)

Check constraints:

"constraint_1" CHECK (t_stamp >= '2024-05-13 16:09:00+00'::timestamp with time zone AND t_stamp < '2024-05-13 16:10:00+00'::timestamp with time zone)

Inherits: t_timescale

Access method: heap

Administering Hypertable CHUNKS

Compressing CHUNKS

```
ex02=# alter table t_timescale
      set (timescaledb.compress,
          timescaledb.compress_orderby = 't_stamp asc');
```

```
ex02=# select * from compress_chunk('_timescaledb_internal._hyper_1_3_chunk',true);
      compress_chunk
```

```
-----
 _timescaledb_internal._hyper_1_3_chunk
(1 row)
```

```
Time: 13221.830 ms (00:13.222)
```

```
ex02=# \dt+ _timescaledb_internal._hyper*chunk
```

List of relations							
Schema	Name	Type	Owner	Persistence	Access method	Size	Description
_timescaledb_internal	_hyper_1_1_chunk	table	postgres	permanent	heap	184 MB	
_timescaledb_internal	_hyper_1_2_chunk	table	postgres	permanent	heap	587 MB	
_timescaledb_internal	_hyper_1_3_chunk	table	postgres	permanent	heap	0 bytes	
_timescaledb_internal	_hyper_1_4_chunk	table	postgres	permanent	heap	677 MB	
_timescaledb_internal	_hyper_1_5_chunk	table	postgres	permanent	heap	704 MB	
_timescaledb_internal	_hyper_1_6_chunk	table	postgres	permanent	heap	703 MB	
_timescaledb_internal	_hyper_1_7_chunk	table	postgres	permanent	heap	84 MB	

(7 rows)

ATTENTION: *administering CHUNKs is not available under apache license*

Administering Hypertable CHUNKS

Compressing CHUNKS

```
ex02=# select * from hypertable_compression_stats('t_timescale');
-[ RECORD 1 ]-----+-----
total_chunks          | 7
number_compressed_chunks | 1
before_compression_table_bytes | 746561536
before_compression_index_bytes | 393396224
before_compression_toast_bytes | 0
before_compression_total_bytes | 1139957760
after_compression_table_bytes | 2129920
after_compression_index_bytes | 0
after_compression_toast_bytes | 404242432
after_compression_total_bytes | 406372352
node_name             |
```

ATTENTION: *administering CHUNKs is not available under apache license*

Administering Hypertable CHUNKS

Decompressing CHUNKS

```
ex02=# select * from decompress_chunk('_timescaledb_internal._hyper_1_3_chunk');
-[ RECORD 1 ]-----+-----
decompress_chunk | _timescaledb_internal._hyper_1_3_chunk

Time: 32983.265 ms (00:32.983)
```

```
ex02=# \dt+ _timescaledb_internal._hyper*chunk
```

List of relations							
Schema	Name	Type	Owner	Persistence	Access method	Size	Description
_timescaledb_internal	_hyper_1_1_chunk	table	postgres	permanent	heap	184 MB	
_timescaledb_internal	_hyper_1_2_chunk	table	postgres	permanent	heap	587 MB	
_timescaledb_internal	_hyper_1_3_chunk	table	postgres	permanent	heap	712 MB	
_timescaledb_internal	_hyper_1_4_chunk	table	postgres	permanent	heap	677 MB	
_timescaledb_internal	_hyper_1_5_chunk	table	postgres	permanent	heap	704 MB	
_timescaledb_internal	_hyper_1_6_chunk	table	postgres	permanent	heap	703 MB	
_timescaledb_internal	_hyper_1_7_chunk	table	postgres	permanent	heap	84 MB	

(7 rows)

ATTENTION: *administering CHUNKs is not available under apache license*

Administering Hypertable CHUNKS

Setting A CHUNK Compression Policy

```
ex02=# -- recall
alter table t_timescale
    set (timescaledb.compress,
        timescaledb.compress_orderby = 't_stamp asc');
ALTER TABLE
```

```
ex02=# -- compresses "t_timescale" chunks that are older than 2 minutes
select add_compression_policy('t_timescale', compress_after => INTERVAL '2m');
add_compression_policy
-----
1000
```

```
ex02=# insert into t_timescale(id) select gen_random_uuid() from generate_series (1, 2E7);
INSERT 0 20000000
```

```
ex02=# select * from pg_size_pretty(hypertable_size('t_timescale'));
pg_size_pretty
-----
3927 MB
(1 row)
```

ATTENTION: *administering CHUNKs is not available under apache license*

Administering Hypertable CHUNKS

Setting A CHUNK Compression Policy

```
ex02=# \dt+ _timescaledb_internal._hyper*chunk
```

List of relations

Schema	Name	Type	Owner	Persistence	Access method	Size	Description
_timescaledb_internal	_hyper_1_14_chunk	table	postgres	permanent	heap	0 bytes	
_timescaledb_internal	_hyper_1_17_chunk	table	postgres	permanent	heap	0 bytes	
_timescaledb_internal	_hyper_1_18_chunk	table	postgres	permanent	heap	0 bytes	
_timescaledb_internal	_hyper_1_1_chunk	table	postgres	permanent	heap	0 bytes	
_timescaledb_internal	_hyper_1_2_chunk	table	postgres	permanent	heap	0 bytes	
_timescaledb_internal	_hyper_1_3_chunk	table	postgres	permanent	heap	0 bytes	
_timescaledb_internal	_hyper_1_4_chunk	table	postgres	permanent	heap	0 bytes	
_timescaledb_internal	_hyper_1_5_chunk	table	postgres	permanent	heap	0 bytes	
_timescaledb_internal	_hyper_1_6_chunk	table	postgres	permanent	heap	0 bytes	
_timescaledb_internal	_hyper_1_7_chunk	table	postgres	permanent	heap	0 bytes	

(10 rows)

```
ex02=# select * from hypertable_compression_stats('t_timescale');
```

```
-[ RECORD 1 ]-----  
total_chunks          | 10  
number_compressed_chunks | 10  
before_compression_table_bytes | 5361410048  
before_compression_index_bytes | 2824945664  
before_compression_toast_bytes | 0  
before_compression_total_bytes | 8186355712  
after_compression_table_bytes | 16318464  
after_compression_index_bytes | 0  
after_compression_toast_bytes | 2902925312  
after_compression_total_bytes | 2919243776  
node_name             |
```

```
ex02=# -- disabling the compression policy is accomplished thusly:
```

```
ex02=# select remove_compression_policy('t_timescale');  
remove_compression_policy
```

```
-----  
t  
(1 row)
```

ATTENTION: *administering CHUNKs is not available under apache license*

Administering Hypertable CHUNKS

Dropping CHUNKS, cont'd

```
ex01=# select show_chunks('t1');
        show_chunks
```

```
-----
 _timescaledb_internal._hyper_1_1_chunk
 _timescaledb_internal._hyper_1_2_chunk
 _timescaledb_internal._hyper_1_3_chunk
 _timescaledb_internal._hyper_1_4_chunk
 _timescaledb_internal._hyper_1_5_chunk
 _timescaledb_internal._hyper_1_6_chunk
 _timescaledb_internal._hyper_1_7_chunk
 _timescaledb_internal._hyper_1_8_chunk
 _timescaledb_internal._hyper_1_9_chunk
 _timescaledb_internal._hyper_1_10_chunk
 _timescaledb_internal._hyper_1_11_chunk
 _timescaledb_internal._hyper_1_12_chunk
 _timescaledb_internal._hyper_1_13_chunk
```

```
begin;
  select drop_chunks('t1', '2024-05-09'::date);
  select show_chunks('t1');
rollback;
```

```
ex01=# \d+ _timescaledb_internal._hyper_1_1_chunk
```

Table "_timescaledb_internal._hyper_1_1_chunk"									
Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description	
id	bigint		not null	nextval('t1_id_seq'::regclass)	plain				
t_stamp	timestamp with time zone		not null	now()	plain				
payload	text				extended				

Indexes:

```
"_hyper_1_1_chunk_t1_t_stamp_idx" btree (t_stamp DESC)
```

Check constraints:

```
"constraint_1" CHECK (t_stamp >= '2024-05-02 00:00:00+00'::timestamp with time zone AND t_stamp < '2024-05-09 00:00:00+00'::timestamp with time zone)
```

Inherits: t1

Access method: heap

Administering Hypertable CHUNKS

CHUNK Function Calls

General Purpose Function Calls

Schema	Name
public	attach_chunk
public	chunk_columnstore_stats
public	chunk_compression_stats
public	chunks_detailed_size
public	compress_chunk
public	decompress_chunk
public	detach_chunk
public	disable_chunk_skipping
public	drop_chunks
public	enable_chunk_skipping
public	merge_chunks
public	merge_chunks
public	merge_chunks_concurrently
public	move_chunk
public	recompress_chunk
public	reorder_chunk
public	set_adaptive_chunking
public	set_chunk_time_interval
public	show_chunks

Compression Function Calls

Schema	Name
public	add_compression_policy
public	chunk_compression_stats
public	hypertable_compression_stats
public	remove_compression_policy

Administering Hypertable CHUNKS

Chunk Runtime Parameters

name	setting
default_toast_compression	pglz
timescaledb.compression_batch_size_limit	1000
timescaledb.compression_orderby_default_function	_timescaledb_functions.get_orderby_defaults
timescaledb.compression_segmentby_default_function	_timescaledb_functions.get_segmentby_defaults
timescaledb.debug_compression_path_info	off
timescaledb.default_chunk_time_interval	
timescaledb.enable_bool_compression	on
timescaledb.enable_bulk_decompression	on
timescaledb.enable_chunk_append	on
timescaledb.enable_chunk_auto_publication	off
timescaledb.enable_chunk_skipping	off
timescaledb.enable_chunkwise_aggregation	on
timescaledb.enable_compression_indexscan	off
timescaledb.enable_compression_ratio_warnings	on
timescaledb.enable_compression_wal_markers	on
timescaledb.enable_decompression_sorted_merge	on
timescaledb.enable_delete_after_compression	off
timescaledb.enable_dml_decompression	on
timescaledb.enable_dml_decompression_tuple_filtering	on
timescaledb.enable_exclusive_locking_recompression	off
timescaledb.enable_hypertable_compression	on
timescaledb.enable_in_memory_recompression	on
timescaledb.enable_parallel_chunk_append	on
timescaledb.enable_rowlevel_compression_locking	off
timescaledb.enable_segmentwise_recompression	on
timescaledb.enable_uuid_compression	on
timescaledb.max_cached_chunks_per_hypertable	1024
timescaledb.max_open_chunks_per_insert	1024
wal_compression	off

- Partitioning

- Choose the right partitioning key

- Choose the optimal CHUNK size

- Maintain consistent partition sizes

- Where possible, enforce data retention policies

- Leverage tablespaces, different storage mediums, as I/O increases

- Optimize your queries (DUH)

- Create partitions in advance (it's understood CPU usage is consistent)

- Determining CHUNK time interval

- $(\text{TOTAL RAM}) \times 25\% = \text{CHUNK time interval}$

- EX: $64\text{GB} \times 0.25 = 16\text{GB}$

MATERIAL NOT COVERED

Moving Forward ...

- TimescaleDB Concepts
 - Schema Management
 - Indexing
 - Constraints
 - Triggers
 - Tablespaces
 - Altering HyperTables
 - Time Buckets
 - Continuous aggregation
 - Configuration: Runtime Parameters
- Querying Data:
 - Convenience function calls
 - PostgreSQL
 - Timescale specific

- Scaling:
 - High Availability
 - High Performance
 - Patroni
 - K8s
- Disaster Recovery
 - Failover
 - Point In Time Recovery (PITR)
- Upgrades/Downgrades
- more ...



Caveat

The extension dilemma

They don't play well together!



PERCONA
for PostgreSQL

Questions?



PERCONA
for PostgreSQL

Thank You :-)